

Matching Engine

Guia de estudo — teoria, vocabulário, decisões de projeto e roteiro de construção

Este documento é para ser lido *antes* de escrever código. Ele não contém implementação: contém o vocabulário, as regras do domínio, as estruturas de dados que sustentam tudo, as decisões que o enunciado deixa em aberto e a ordem em que convém construir. Os poucos trechos de Python que aparecem são mecânica da linguagem em contexto neutro, nunca a solução. O objetivo é que, ao sentar para codar, você já não esteja aprendendo conceito — só traduzindo para a linguagem.

1. O vocabulário

Metade do tempo perdido num exercício desses vem de não ter os termos firmes. Esta tabela é a referência para o resto do documento.

Termo	Significado
Ordem	Manifestação de interesse em comprar ou vender. Tem lado, quantidade e — se for limit — preço.
Lado (<i>side</i>)	<code>buy</code> ou <code>sell</code> . Determina contra qual metade do livro a ordem tenta casar.
Quantidade (<i>qty</i>)	Número de ações. Sempre inteiro.
Livro de ofertas	Estrutura que guarda as ordens ainda não executadas , separadas por lado e ordenadas por prioridade.
Bid	O melhor preço de compra do livro — o mais alto que alguém está disposto a pagar.
Ask / Offer	O melhor preço de venda — o mais baixo que alguém aceita receber.
Spread	Ask – Bid. O vão entre os dois lados. Se ele fecha ou inverte, há negócio.
Nível de preço	Todas as ordens de um mesmo lado a um mesmo preço. Dentro dele vale ordem de chegada.
Profundidade	Quantidade total disponível em cada nível.
Ordem passiva (<i>resting, maker</i>)	A que já estava no livro esperando. É ela que define o preço do trade.
Ordem agressora (<i>aggressive, taker</i>)	A que acaba de chegar e consome liquidez existente.
Marketable	Ordem que, ao chegar, já cruza com o outro lado — executa em vez de repousar.
Cruzamento	A decisão : os preços são compatíveis? (compra $\geq$ venda)
Execução / Trade	O efeito : transferir quantidade, decrementar as ordens, remover as zeradas, imprimir.
Preenchimento parcial	Quando só parte da quantidade executa. O resto continua (ou é descartado, se market).
Prioridade preço-tempo	Primeiro o melhor preço; empatou no preço, ganha quem chegou antes.
Price improvement	Quando o agressor recebe preço melhor que o seu limite. Consequência da regra do preço passivo.
Livro cruzado	Estado inválido : bid $\geq$ ask. Nunca pode persistir — se acontecer, há bug.
Ordem pegged	Ordem sem preço próprio: acompanha uma referência (o bid ou o offer), repreçada pela engine.

2. Anatomia dos comandos

A gramática vem dos exemplos do enunciado. Repare na **ordem dos números** — é a fonte de erro mais boba possível.

```

limit <side> <price> <qty>      limit buy 10 100
market <side> <qty>              market buy 150
peg <bid|offer> <side> <qty>    peg bid buy 150
cancel order <id>                cancel order identificador_1
print book

```

O PREÇO VEM ANTES DA QUANTIDADE

`limit buy 10 100` significa **comprar 100 ações ao preço de 10** — e não comprar 10 ao preço de 100. O próprio enunciado confirma na saída do requisito 3: `Order created: buy 100 @ 10`. A ordem market não tem preço, então `market buy 150` tem um número só: a quantidade.

Saídas obrigatórias

```
Trade, price: <preço>, qty: <quantidade>
Order created: <side> <qty> @ <price> <id>
Order cancelled
```

O formato do `Trade` é literal e será conferido. Os outros dois aparecem só no exemplo do requisito 3 — o enunciado não diz se `Order created` deve sair sempre ou só em alguns casos. **Decida e registre no README.** A sintaxe da alteração de ordem também não é dada: você escolhe.

3. O modelo mental: orientado a eventos

Não existe relógio. Não existe laço de fundo, thread ou timer. O sistema fica **bloqueado esperando um comando**; quando chega, processa, imprime e volta a esperar. Entre dois comandos, o livro é uma fotografia congelada — nada expira, nada decai.

Formalmente, a engine é uma função:

```
(estado do livro, comando) → (novo estado, saída)
```

Rodando o exemplo do enunciado, estado por estado:

#	Comando	Compras	Vendas	Saída
1	<code>limit buy 10 100</code>	100 @ 10	—	
2	<code>limit sell 20 100</code>	100 @ 10	100 @ 20	
3	<code>limit sell 20 200</code>	100 @ 10	100 @ 20 · 200 @ 20	
4	<code>market buy 150</code>	100 @ 10	150 @ 20	Trade, price: 20, qty: 150
5	<code>market buy 200</code>	100 @ 10	—	Trade, price: 20, qty: 150
6	<code>market sell 200</code>	—	—	Trade, price: 10, qty: 100

Três leituras que os exemplos **fixam**, e que você não precisa inventar:

- **Market não repousa.** No passo 5 havia só 150 disponíveis para uma ordem de 200; executou 150 e os outros 50 sumiram. Ordem a mercado nunca vira ordem no livro.
- **Trades saem agregados por nível.** No passo 4 duas ordens distintas foram consumidas (100 e 50) e saiu **uma única linha** com qty 150. Exchange real emitiria dois trades. Escolha e justifique.
- **Compra a 10 e venda a 20 não casam.** Ninguém vende a 10 nem compra a 20 — as duas repousam e o spread fica aberto.

4. As regras do casamento

Quatro perguntas, sempre nesta ordem.

Contra qual lado?

Uma compra casa contra as vendas; uma venda casa contra as compras. Sempre varrendo **do melhor para o pior**: as vendas da mais barata para a mais cara, as compras da mais cara para a mais barata.

O preço é aceitável?

Este é o **único** ponto onde limit e market diferem — e a diferença cabe em uma linha:

Tipo	Critério de aceitação
Market	Sempre aceita. Não há preço a comparar.
Limit buy	Aceita enquanto o preço da venda for $\leq$ ao seu limite.
Limit sell	Aceita enquanto o preço da compra for $\geq$ ao seu limite.

Não escreva dois algoritmos. Escreva um, com esse teste parametrizado — uma market é apenas uma limit com limite infinito.

LIMIT É LIMITE, NÃO ALVO

Uma `limit buy 25` significa "**compre a 25 ou menos**", não "compre a 25". Se houver venda a 20, o trade sai a 20 e você economizou. Uma `limit sell 20` significa "**venda a 20 ou mais**". O número declarado é o **pior** preço que você tolera.

A que preço executa?

Ao preço da ordem passiva — a que já estava no livro. Ela publicou seus termos primeiro; o agressor aceitou o que estava anunciado. Por isso a compra a 25 executa a 20: quem chegou depois recebe a melhora.

Que quantidade?

O **mínimo** entre o que resta da agressora e o que resta da passiva. Por isso uma ordem grande gera várias execuções, e pode atravessar vários níveis:

Livro de vendas `100 @ 20`, `100 @ 22`, `100 @ 26`; chega `limit buy 25 250`:

Nível	25 aceita?	Resultado
100 @ 20	sim, $20 \leq 25$	Trade, price: 20, qty: 100
100 @ 22	sim, $22 \leq 25$	Trade, price: 22, qty: 100
100 @ 26	não , $26 > 25$	para — os 50 restantes repousam a 25

Cada nível negocia **ao próprio preço**, não ao preço da agressora. É isso que "varrer o livro" significa.

5. Prioridade preço-tempo

Dois critérios, nesta ordem:

1. **Preço.** Melhor preço executa primeiro, sempre.

2. **Tempo.** Empatou no preço, ganha quem chegou antes — fila FIFO dentro do nível.

"TEMPO" NÃO É HORA DO RELÓGIO

Use um **contador monotônico** incrementado a cada ordem aceita, não `datetime.now()`. Dois eventos podem cair no mesmo microssegundo, e o contador ainda deixa o sistema determinístico — o mesmo roteiro de comandos produz sempre o mesmo resultado, que é o que torna os testes possíveis.

6. Nós, referências e filas

Esta seção é pré-requisito da próxima. Com estes três conceitos firmes, a estrutura de dados do livro sai quase sozinha.

Por que não usar uma lista comum

A tentação é guardar cada nível como uma `list` do Python e seguir em frente. O problema aparece no cancelamento:

Operação numa <code>list</code>	Custo
Inserir no fim — <code>append</code>	$O(1)$ — ótimo
Remover do começo — <code>pop(0)</code>	$O(N)$ — desloca todos os outros
Achar a ordem de id X	$O(N)$ — varre até encontrar
Remover do meio	$O(N)$ — acha e depois desloca

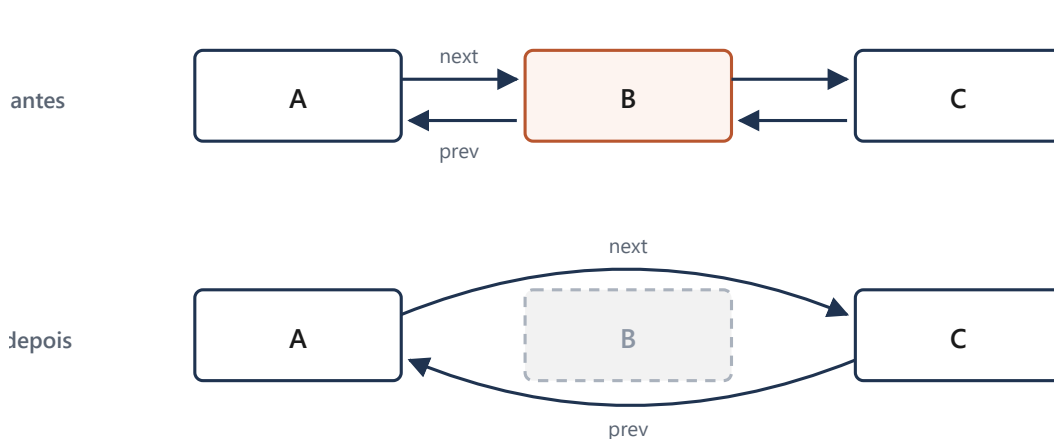
Como o cancelamento é obrigatório e pode cair em qualquer posição da fila, a lista comum te condena a $O(N)$. É por isso que existe a lista encadeada.

O que é um nó

Um **nó** é um objeto que guarda duas coisas: o dado em si e **quem são seus vizinhos**. Ele não sabe sua posição, não sabe o tamanho da fila, não conhece nada global — só os dois vizinhos.

Numa lista **simplesmente** encadeada cada nó conhece apenas o próximo. Para remover um nó do meio você precisa do *anterior*, e a única forma de achá-lo é percorrer desde a cabeça — $O(N)$ de novo, e você não ganhou nada.

Numa lista **duplamente** encadeada cada nó conhece o anterior e o próximo. Aí a remoção dispensa busca: basta costurar os dois vizinhos um no outro.



B saiu da fila sem que ninguém precise ser percorrido ou deslocado. A e C passam a se apontar diretamente.

```
B.prev.next = B.next      # A passa a apontar para C
B.next.prev = B.prev      # C passa a apontar para A
B.prev = B.next = None    # B sai da fila
```

Três atribuições, sem laço e sem busca: **O(1)**. É esta operação — e só ela — que justifica toda a complexidade da estrutura.

CUIDADO COM AS PONTAS

Se B for a cabeça da fila, `B.prev` é `None` e a primeira linha estoura; se for a cauda, a segunda estoura. Trate os dois casos, ou use **nós-sentinela** de cabeça e cauda — nós falsos que existem só para nunca haver ponta nula. É o truque clássico e elimina vários `if` espalhados.

A fila FIFO

Cada nível de preço é uma fila: entra pela **cauda**, sai pela **cabeça**. Guardando as duas pontas, ambas as operações são $O(1)$ — e isso é exatamente o que a prioridade por tempo pede. A ordem mais antiga do nível é sempre a cabeça; a que acabou de chegar é sempre a cauda. Você nunca precisa ordenar nada dentro do nível: a estrutura já nasce ordenada por chegada.

Python não tem ponteiro — tem referência

Em C você guardaria um endereço de memória. Em Python toda variável já é uma **referência** a um objeto — um nome colado numa etiqueta. Dois nomes podem apontar para o **mesmo** objeto:

```
a = {"nome": "X"}
b = a                # b NÃO é uma cópia
b["nome"] = "Y"
print(a["nome"])     # -> "Y"
```

Fora deste contexto isso costuma ser fonte de bug. Aqui é **precisamente o que você quer**: é o que faz o índice `id → ordem` funcionar. O dicionário e a fila do nível guardam o **mesmo objeto**, não duas cópias. Você localiza a ordem em $O(1)$ pelo dicionário, decrementa a quantidade ali, e a fila enxerga a mudança no mesmo instante — porque não há duas coisas, há uma.

A REGRA PRÁTICA

Nunca copie uma ordem. Existe **um** objeto por ordem, e todos os índices apontam para ele. No dia em que existirem duas cópias, elas vão divergir — e esse é o tipo de bug que consome uma tarde.

7. As estruturas de dados

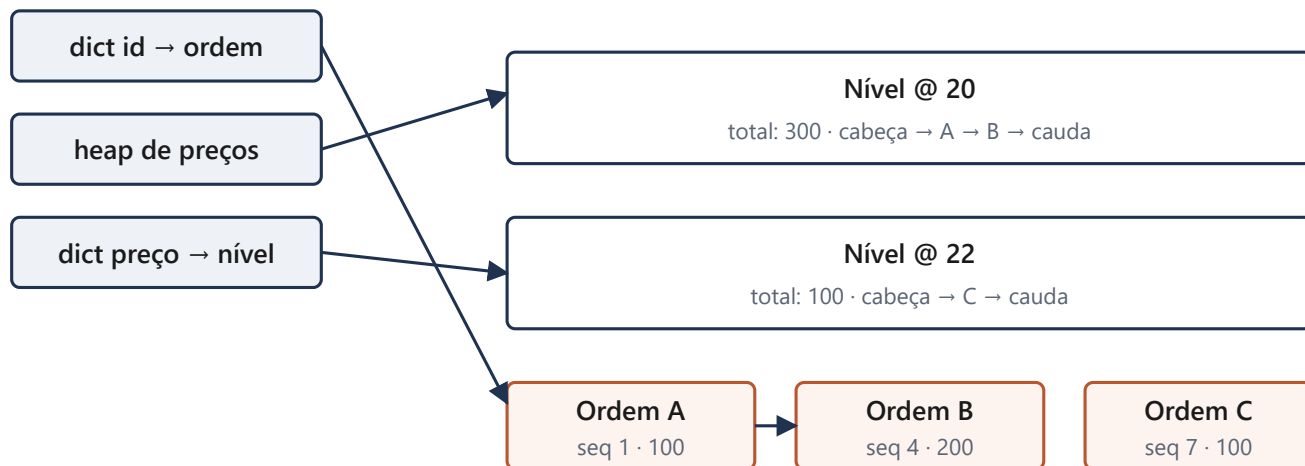
A ideia central: **três índices sobre os mesmos objetos**, cada um respondendo a uma pergunta diferente em tempo bom. Escolher isso certo *antes* de codar é o que evita a refatoração que come um dia inteiro.

Índice	Responde	Custo
dict preço → nível	"existe nível neste preço?"	$O(1)$
dict id → ordem	"onde está a ordem X?" (cancelar, alterar)	$O(1)$
heap ou mapa ordenado	"qual o melhor preço deste lado?"	$O(\log P)$
lista duplamente encadeada	"quem é o próximo da fila deste nível?"	$O(1)$

POR QUE DUPLAMENTE ENCADEADA

Para cancelar uma ordem do **meio** da fila sem varrer a fila inteira, você precisa do ponteiro para o anterior — só assim dá para costurar os vizinhos e sair em $O(1)$. Com lista simples ou com array, o cancelamento vira $O(N)$ e você perde o requisito de complexidade.

Sobre o heap: quando um nível esvazia, **não tente removê-lo do heap**. Use remoção preguiçosa — deixe o preço morto lá e descarte quando ele chegar ao topo e o dicionário disser que o nível não existe mais. É mais simples e continua correto. Em Python, `SortedDict` resolve o mesmo problema de forma mais legível. Qualquer uma serve; saiba defender a escolha.



Os três índices apontam para as mesmas ordens — nada é duplicado, o que varia é o caminho de acesso. As ordens A e B formam a fila do nível @ 20; C é a única do nível @ 22.

8. Caixa de ferramentas da `stdlib`

O que a biblioteca padrão já resolve, para você não reinventar. Nenhuma dependência externa é necessária — a única opcional está na última linha.

Peça	Para quê	Observação
<code>dict</code>	id → ordem · preço → nível	Busca e inserção O(1). É a base dos dois índices diretos.
<code>heapq</code>	melhor preço de cada lado	Só existe min -heap — ver a caixa abaixo.
<code>dataclasses</code>	objetos de domínio	Gera <code>__init__</code> e <code>__repr__</code> . O <code>__repr__</code> automático ajuda muito na depuração; <code>slots=True</code> economiza memória.
<code>enum.Enum</code>	lado e tipo de ordem	Evita string solta — <code>"BUY"</code> digitado errado passa silencioso, <code>Side.BUY</code> não.
<code>itertools.count</code>	nº de sequência e ids	Contador monotônico e determinístico. Melhor que relógio ou <code>uuid</code> para poder testar.
<code>decimal.Decimal</code>	preço, se não usar inteiro	Alternativa ao inteiro em centavos: mais lento, porém mais legível.
<code>typing</code>	anotações de tipo	Não muda o comportamento, mas documenta e o editor avisa antes de rodar.
<code>unittest</code> · <code>pytest</code>	testes	<code>unittest</code> já vem junto; <code>pytest</code> é mais enxuto de escrever.
<code>random</code>	teste de fuzz	Fixe a semente, senão a falha não se reproduz.
<code>sortedcontainers</code>	alternativa ao heap	Externa. <code>SortedDict</code> mantém as chaves ordenadas e dispensa a remoção preguiçosa.

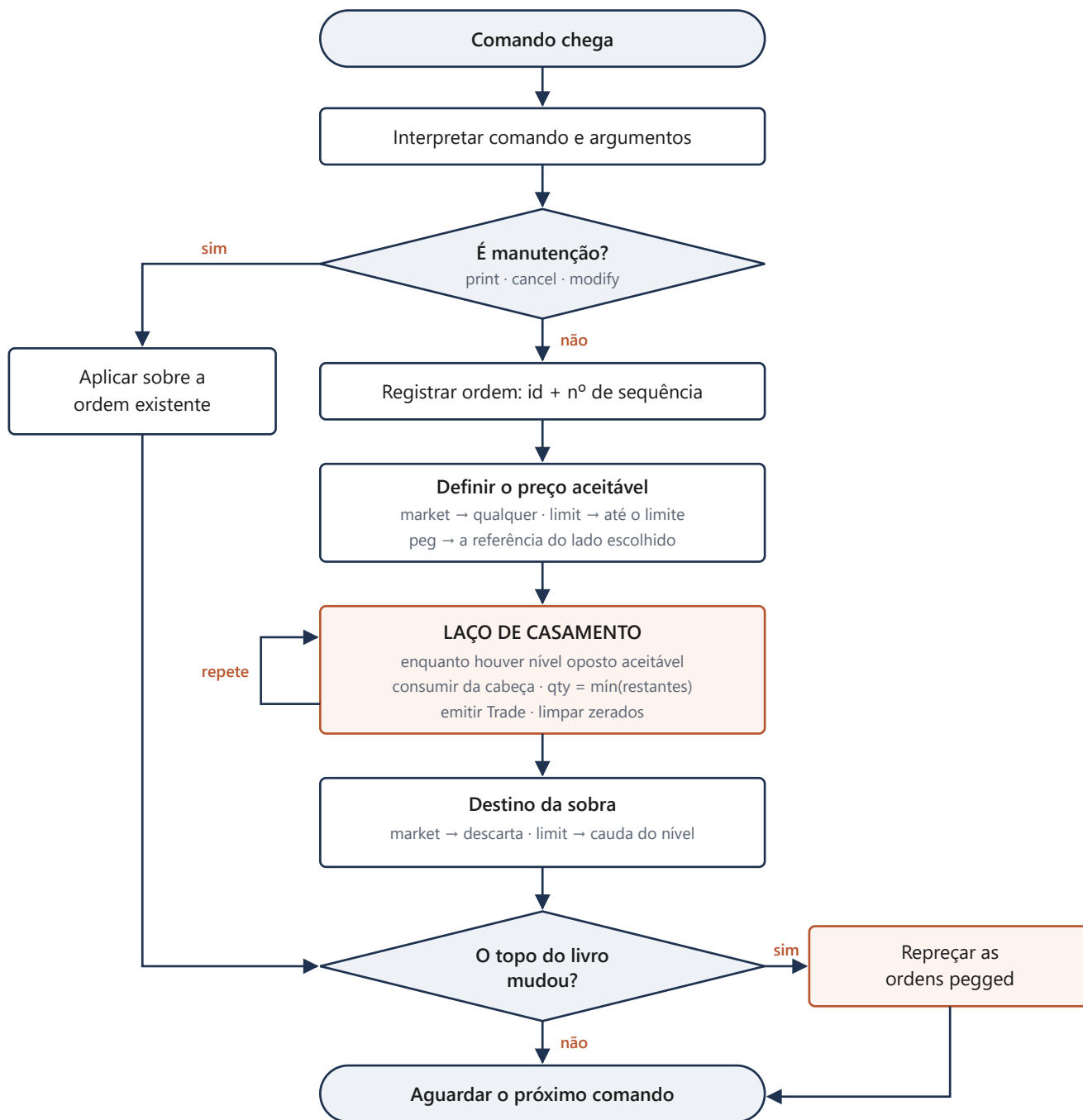
HEAPQ SÓ FAZ MIN-HEAP

Para o lado das compras você precisa do **maior** preço, mas o `heapq` só entrega o menor. O truque padrão é guardar o preço **negado**:

```
import heapq
h = []
heapq.heappush(h, -preco)  # guarda negativo
maior = -h[0]              # desfaz o sinal ao ler
```

Isole a negação num único ponto do código. Sinal trocado espalhado por várias funções é receita de bug difícil.

9. Fluxo de um comando



Ciclo completo. Repare que a repreçagem das pegged é o último passo de **qualquer** operação que possa mexer no topo — inclusive cancelamento e alteração.

10. Os requisitos, um a um

Visualização do livro

Duas colunas, compras à esquerda ordenadas do maior preço para o menor, vendas à direita do menor para o maior. Cada linha é uma ordem (não um nível agregado) — só assim dá para **ver a fila** e conferir a prioridade, que é o que os exemplos do enunciado mostram.

Cancelamento

Recebe um id, encontra a ordem pelo dicionário, costura os vizinhos na lista encadeada, decrementa o total do nível e remove o nível se ele esvaziou. Tudo $O(1)$, exceto a eventual limpeza do heap.

Decida o que fazer com id inexistente ou já cancelado: erro, mensagem, silêncio.

Alteração

A regra que o enunciado dá: **mudar o preço faz perder a prioridade** — a ordem vai para a cauda do novo nível. O enunciado **não** diz o que acontece ao mudar só a quantidade. A regra de mercado real, que vale adotar e citar:

Alteração	Prioridade
Preço muda	Perde — vai para a cauda do novo nível
Quantidade aumenta	Perde — está pedindo mais do que a fila reservou
Quantidade diminui	Mantém — reduzir não prejudica ninguém atrás

Pense na implementação como **remover e reinserir**, e não como editar no lugar. Fica mais simples e naturalmente correto.

11. Ordens pegged — o ponto mais difícil

Uma pegged não tem preço próprio: ela **acompanha** uma referência. `peg bid buy` fica sempre no melhor preço de compra; se o bid subir, ela sobe junto.

ELA NÃO VIGIA NADA

Parece que precisa de monitoramento contínuo. Não precisa. O melhor preço só muda quando um **evento** o muda, e são só quatro:

- chega uma limit melhor que o topo atual;
- a ordem do topo é **cancelada**;
- a ordem do topo é **alterada** de preço;
- o nível de topo é **consumido** por um agressor.

Em todos, você já está dentro da operação. Basta comparar o topo antes e depois e, se mudou, repreçar. É um passo síncrono, não uma thread.

Três decisões obrigatórias

A referência ignora as próprias pegged. Se elas entrassem no cálculo, repreçar uma poderia mover o topo, o que dispararia outra repreçagem — cascata potencialmente infinita. Calculando a referência só sobre as ordens comuns, o gatilho é sempre externo e o processo termina em um passo. Isso também resolve o caso do livro sem nenhuma ordem comum: a pegged simplesmente não tem preço e fica de fora até existir referência.

Uma pegged pode cruzar. Um `peg offer buy` sentaria no preço da melhor venda, ou seja, cruzando na hora. Decida: executa, ou trava no melhor preço passivo do próprio lado? Escreva a escolha no README.

A CONTRADIÇÃO APARENTE ENTRE OS REQUISITOS 4 E 5

O requisito 4 diz que mudar de preço **perde prioridade**. Mas no exemplo do requisito 5, quando chega `limit buy 10.1 300`, a pegged de 150 é repreçada para 10,1 e aparece **à frente** da ordem nova:

```
150 @ 10.1    ← a pegged, repreçada
300 @ 10.1    ← a limit que acabou de chegar
```

Os dois requisitos só se conciliam se você distinguir **quem causou** a mudança: no requisito 4 foi o *cliente*, então é justo ir para o fim da fila; na pegged foi a *engine*, e penalizar o cliente por uma ação que não foi dele não faz sentido. Implemente seguindo o exemplo (a pegged preserva o timestamp original) e **escreva essa justificativa**. Perceber isso vale mais que qualquer refinamento de performance.

12. Decisões que você precisa justificar

O enunciado diz literalmente que o comportamento escolhido deve ser justificado. Cada linha desta tabela é um parágrafo do seu README.

Questão	Recomendação	Por quê
Limit que cruza: ignorar ou preencher?	Preencher	É o que exchange real faz, e unifica o código — market vira caso particular de limit.
Market sem liquidez suficiente	Executa o possível, descarta o resto	Fixado pelos exemplos do enunciado (passos 5 e 6).
Trade por nível ou por contraparte?	Por nível	É o que o exemplo mostra. Real seria por contraparte — cite a diferença.
Alterar quantidade perde prioridade?	Aumentar sim, diminuir não	Regra de mercado real; reduzir não prejudica quem está atrás.
Pegged perde prioridade ao repreçar?	Não	Quem repreçou foi a engine, não o cliente. Segue o exemplo do requisito 5.
Referência inclui as pegged?	Não	Evita cascata de repreçagem.
Pegged que cruzaria	Trava no lado passivo	Preserva a natureza passiva da ordem. Alternativa defensável: executar.
Representação do preço	Inteiro em centavos	Float quebra comparação de igualdade.

13. Armadilhas

PREÇO EM PONTO FLUTUANTE

Os exemplos usam 9.99, 10.1, 10.5. Comparar float por igualdade vai te morder — $0.1 + 0.2 \neq 0.3$. Guarde o preço como **inteiro em centavos** (ou `Decimal`) e converta só na exibição. Barato de fazer no começo, caríssimo depois.

- **Esquecer de limpar o nível vazio.** Ordem zerada tem que sair da fila, e nível sem ordens tem que sair do dicionário. Senão o "melhor preço" passa a apontar para o vazio.
- **Esquecer de decrementar o total do nível.** A quantidade agregada tem que bater com a soma das ordens — é um invariante que vale testar.
- **Deixar market repousar no livro.** Não existe ordem a mercado no livro. Nunca.
- **Amarrar a engine ao `input()`.** Mantenha a lógica separada da interface: os testes chamam métodos diretamente, sem simular digitação.
- **Livro cruzado.** Se depois de alguma operação o melhor bid ficar $\geq$ ao melhor ask, há bug. Vale checar isso como asserção.

14. Testes

É onde se ganha nota barato. Três camadas:

- 1 **Os exemplos do enunciado, transcritos literalmente.** São a especificação. Se não passam, nada mais importa.
- 2 **Invariantes, checados após toda operação:** o livro nunca está cruzado; o total de cada nível bate com a soma das ordens; todo id no índice aponta para uma ordem que está de fato em algum nível; nenhuma ordem tem quantidade zero ou negativa.
- 3 **Fuzz.** Gere milhares de operações aleatórias — inserir, cancelar, alterar, pegged — e afirme os invariantes a cada passo. Custa pouco e acha os bugs de canto que teste manual não acha.

15. Roteiro de construção

Esta ordem é também o seu histórico Git, que **faz parte da avaliação**. Um commit por comportamento, com teste junto — nunca tudo num commit final.

- 1 **Tipos de domínio e livro vazio.** Ordem, nível, livro. Mais a visualização, que você vai usar para depurar todo o resto.
- 2 **Limit orders que só repousam.** Sem casamento ainda. Confirme que a ordenação de preço e a fila FIFO dentro do nível estão certas.
- 3 **Casamento.** O laço, o preço passivo, a quantidade mínima, a varredura de níveis. Aqui aparece o primeiro `Trade`.
- 4 **Market orders.** Deve ser quase de graça — só o critério de aceitação e o descarte da sobra.
- 5 **Cancelamento.** É o teste real da sua estrutura de dados: se não sair $O(1)$, volte ao passo 1.
- 6 **Alteração.** Remover e reinserir, com as regras de prioridade.
- 7 **Pegged.** O gatilho de repreçagem e as três decisões da seção 11.
- 8 **CLI.** Leitura de linha, parser, despacho. Fina, sem lógica de negócio.
- 9 **README.** Cada linha da tabela da seção 12 vira um parágrafo. Mais a análise de complexidade e as limitações conhecidas.

ANTES DE ABRIR O EDITOR

Gaste as duas ou três primeiras horas **no papel**: desenhe o livro com dois níveis de cada lado, três ordens em um deles, e simule à mão um cancelamento no meio da fila e uma alteração de preço. Se o desenho aguentar essas duas operações sem varrer nada, a estrutura está certa. Se não aguentar, é muito mais barato descobrir agora.

16. Checklist final

- ☐ Os três exemplos do enunciado reproduzem exatamente a saída esperada
- ☐ O formato `Trade, price: X, qty: Y` está literal
- ☐ Cancelamento e alteração funcionam em ordem do meio da fila
- ☐ Alteração de preço recoloca a ordem na faixa certa e no fim da fila
- ☐ Pegged acompanha o bid/offer nos quatro gatilhos
- ☐ Preço não é float
- ☐ Engine separada da interface

- ☐ Invariantes testados; fuzz rodando
- ☐ Histórico Git incremental, com mensagens explicando decisões
- ☐ README com todas as justificativas e a análise de complexidade
- ☐ Você consegue explicar cada linha do código de cabeça